# Session 06 - Employee and Document Classes - Tests

Python Contoso Vault - 6 December 2026

## Part 1 - Core part A: models.py

### Goal

Turn a document and an employee into objects. Make a document guard its own title, so it can never be empty. Give each kind of vault error its own exception, with one shared parent, so one `except` can catch any of them.

Open the Python shell in the `06-employee-and-document-classes` folder and run `from models import *`, then make each call in the test plan.

### Provided

Copy your own `caesar.py`, `accounts.py` and `vault.py` from your Session 05 folder into this folder, and build `models.py` beside them.

### Instructions

1. Import `decrypt` from `caesar.py` and `check_login` from `accounts.py`.
2. Define an exception class `VaultError` that inherits from `Exception`.
3. Define the exception classes `ValidationError`, `AccessDenied` and `NotFound`, each inheriting from `VaultError`.
4. Define a class `Document`.
   4.1. Define a constructor that takes a document ID as an integer, a title as a string, a body as a string and a check as a string.
      4.1.1. Store each value on the object.
   4.2. Define `title` as a property that returns the title.
   4.3. Define its setter, which takes a title as a string.
      4.3.1. Raise `ValidationError` with the message `Title must not be empty.` if the title is empty.
      4.3.2. Otherwise, store the title.
   4.4. Define a method `check_key` that takes a key as an integer.
      4.4.1. Decrypt the check with the key.
      4.4.2. Return `True` if the result is `"CONTOSO-VAULT"`, and `False` otherwise.
5. Define a class `Employee`.
   5.1. Define a constructor that takes an employee ID as a string, a name as a string and a username as a string.
      5.1.1. Store each value on the object.
   5.2. Define a method `check_password` that takes the accounts as a dictionary and a password as a string.
      5.2.1. Return whether the login is right for the employee's username and the password.

### Constraints

- A `Document` object has the attributes `document_id`, `title`, `body` and `check`. An `Employee` object has the attributes `employee_id`, `name` and `username`.
- Assigning an empty title raises `ValidationError` with the message `Title must not be empty.`
- The constructor raises `ValidationError` for an empty title too.
- A title is empty only when it is `""`. A title of spaces is valid.
- A failed title assignment leaves the old title in place.
- The exception message is the text `str(error)` gives.
- No method prints or reads input when called.
- Importing the file reads no input, prints nothing and writes no file.

### Design

This is advice on how to build the code well. No test checks it, and your explain-back looks at it.

- Make `title` a property, with a setter that checks the title.

- Store the title in `_title`. Keep every other attribute public.
- Store the title in the constructor through the property.
- Check the password with `check_login`.
- Give every variable and function a type hint.

## Input validation

Required for the title only, as the constraints set out. Assume every other value has the type the constructor names.

## Test plan

`document_1` is `Document(1, "Payroll", "Vdodulhv jr rxw rq 28 Qryhpehu.", "FRQWRVR-YDXOW")`. Likewise, `nimali` is `Employee("E1001", "Nimali Perera", "nimali")`.

`accounts` is `{"nimali": nimali_hash}`. Here, `nimali_hash` is `31ba6993336b8fbd391af9cac71bb3928077583c9c13c595ea61dfc3b7a576fa`, the hash of `"lotus-2026"`.

| Test | Call or check | Expected output | Actual output | Pass |
|---|---|---|---|---|
| 1 | The attributes of `document_1` | `document_id` `1`, `title` `"Payroll"`, `body` `"Vdodulhv jr rxw rq 28 Qryhpehu."`, `check` `"FRQWRVR-YDXOW"` | | |
| 2 | `document_1.check_key(3)` | `True` | | |
| 3 | `document_1.check_key(4)` | `False` | | |
| 4 | `document_1.title = "Payroll 2026"`, then `document_1.title` | `"Payroll 2026"` | | |
| 5 | `document_1.title = ""` | Raises `ValidationError` with the message `Title must not be empty.` `document_1.title` is still `"Payroll"` | | |
| 6 | `Document(2, "", "Ergb whaw", "FRQWRVR-YDXOW")` | Raises `ValidationError` with the message `Title must not be empty.` | | |
| 7 | `Document(3, "   ", "", "FRQWRVR-YDXOW").title` | `"   "` | | |
| 8 | The attributes of `nimali` | `employee_id` `"E1001"`, `name` `"Nimali Perera"`, `username` `"nimali"` | | |
| 9 | `nimali.check_password(accounts, "lotus-2026")` | `True` | | |
| 10 | `nimali.check_password(accounts, "Lotus-2026")` | `False` | | |
| 11 | `nimali.check_password({}, "lotus-2026")` | `False` | | |
| 12 | `Employee("E1002", "Kasun Silva", "kasun").check_password(accounts, "lotus-2026")` | `False` | | |
| 13 | `issubclass(VaultError, Exception)` | `True` | | |
| 14 | `issubclass(X, VaultError)` for `ValidationError`, `AccessDenied` and `NotFound` | `True` for each | | |
| 15 | `issubclass(AccessDenied, ValidationError)` and `issubclass(NotFound, AccessDenied)` | `False` for each | | |
| 16 | `raise NotFound("No such document.")` inside `try`, caught by `except VaultError as error` | Caught. `str(error)` is `"No such document."` | | |
| 17 | `import models` | Reads no input and prints nothing | | |

## Done criteria

☐ Every test plan row gives exactly the expected result.

# Part 2 - Core part B: vault.py

## Goal

Keep each document in `vault.py` as a `Document` object in place of a dictionary. Leave the file on disk as it is. When something goes wrong, raise a vault error, then catch every vault error in one place in the menu and print its message. Keep everything from Session 05 working, and refuse an empty title.

Run `python vault.py` in the terminal from the `06-employee-and-document-classes` folder and type each input line, then press Enter. Delete `accounts.json` and `documents.json` before each test that starts in an empty folder. To try one function, open the Python shell in the same folder and run `from vault import find_document`.

## Provided

Copy your own `caesar.py`, `accounts.py` and `vault.py` from your Session 05 folder into this folder, and build `models.py` beside them first. `vault.py` already defines `load_documents`, `save_documents`, `create_document`, `check_key` and `search_documents`, and runs the account menu and the five-line document menu.

## Instructions

Amend the copied `vault.py`. Keep the account menu and the file format of `documents.json` and `accounts.json` as they are, and make these changes in the same file.

1. Import `Document`, `VaultError`, `AccessDenied` and `NotFound` from `models.py`.
2. Change `load_documents` to return the documents as a list of `Document` objects.
   2.1. Build one `Document` from each stored dictionary.
   2.2. Return an empty list if the file does not exist.
3. Change `save_documents` to take the documents as a list of `Document` objects.
   3.1. Write each document as a dictionary with the keys `id`, `title`, `body` and `check`.
4. Change `create_document` to build and return a `Document` object.
   4.1. Add the document to the list only after the object is built.
5. Remove the `check_key` function. Use the `Document` method in its place.
6. Change `search_documents` to take and return a list of `Document` objects.
7. Define a function `find_document` that takes the documents as a list and a document ID as an integer.
   7.1. Return the document with that ID.
   7.2. Raise `NotFound` with the message `No such document.` if no document has that ID.
8. When the file runs as a program:
   8.1. Handle each document menu choice inside one `try`.
      8.1.1. On `VaultError`, print the error's message.
   8.2. On `1`, read a title, a body and a key as before, then create the document.
   8.3. On `3`:
      8.3.1. Read a document ID as an integer.
      8.3.2. Find the document.
      8.3.3. Read each key as before.
      8.3.4. After the third wrong key, raise `AccessDenied` with the message `Too many attempts.`
   8.4. In List, Read and Search, read each document's ID, title, body and check from its attributes.
   8.5. Keep every other step of the Session 05 program.

## Constraints

- Every constraint in the Session 05 `vault.py` test plan still holds, except the empty title rule below.
- `documents.json` holds the same JSON as in Session 05: a list of dictionaries with the keys `id`, `title`, `body` and `check`.
- `load_documents`, `create_document`, `search_documents` and `find_document` return `Document` objects. No function returns a document as a dictionary.
- After the third wrong key, the program prints `Too many attempts.`

- Create reads the title, the body and a valid key before it checks the title. An empty title prints `Title must not be empty.`, saves nothing and uses up no ID.
- `find_document` neither prints nor reads input.
- Importing the file reads no input, prints nothing and writes no file.

## Design

This is advice on how to build the code well. No test checks it, and your explain-back looks at it.

- Every Design point in the Session 05 `vault.py` test plan still applies.
- Keep every document in memory as a `Document` object.
- Catch vault errors with exactly one `except VaultError`.
- Write no `except` that names `ValidationError`, `AccessDenied` or `NotFound`.
- Raise `AccessDenied` after the third wrong key, and let the one `except VaultError` print its message.
- Define no `check_key` function. Call the `Document` method in its place.
- Give every variable and function a type hint.

## Input validation

Required for every key and ID, as in Session 05, and for the title through `Document`. Not required for usernames, passwords, bodies and search texts. Assume `documents.json` and `accounts.json` are missing or were written by this program.

## Test plan

### Function tests

`dictionary_1` is `{"id": 1, "title": "Payroll", "body": "Vdodulhv jr rxw rq 28 Qryhpehu.", "check": "FRQWRVR-YDXOW"}`.

Likewise, `dictionary_2` is `{"id": 2, "title": "Kandy rent", "body": "Vlyoj zqqtnp cpye td Cd 150,000.", "check": "NZYEZDZ-GLFWE"}`.

Likewise, `dictionary_3` is `{"id": 3, "title": "Galle rent", "body": "Lfqqj xmtu wjsy nx Wx 90,000.", "check": "HTSYTXT-AFZQY"}`.

`document_1` to `document_3` are the `Document` objects built from them. Here, `documents` is `[document_1, document_2, document_3]`, and `path` names a file in an empty folder. "The same as" means equal values for all four attributes.

| Test | Call | Expected output | Actual output | Pass |
|---|---|---|---|---|
| 1 | `create_document(new_documents, "Payroll", "Salaries go out on 28 November.", 3)`, with `new_documents` as `[]` | A `Document` the same as `document_1`. `new_documents` holds it | | |
| 2 | Then `create_document(new_documents, "", "Body text", 3)` | Raises `ValidationError`. `new_documents` still holds one document | | |
| 3 | `save_documents([document_1, document_2], path)` | The file holds the JSON list `[dictionary_1, dictionary_2]` | | |
| 4 | Then `load_documents(path)` | A list of two `Document` objects, the same as `document_1` and `document_2` | | |
| 5 | `load_documents(path)`, with no file at `path` | `[]` | | |
| 6 | `find_document(documents, 2)` | `document_2` | | |
| 7 | `find_document(documents, 0)` | Raises `NotFound` with the message `No such document.` | | |
| 8 | `find_document(documents, 4)` | Raises `NotFound` with the message `No such document.` | | |
| 9 | `find_document([], 1)` | Raises `NotFound` with the message `No such document.` | | |
| 10 | Session 05 function tests 1 to 9, with each dictionary replaced by its `Document` | The same documents, in the same order, as each Session 05 expected result | | |

**Program tests**

Each test runs `python vault.py` in an empty folder, once or twice in a row. "Sign up Nimali", "Log in Nimali", "Create Payroll" and "Create Kandy rent" are as in the Session 05 `vault.py` test plan. "Create empty" stands for the four input lines `"1"`, empty line, `"Body text"`, `"3"`. Every run prints the account menu and the document menu before each choice, as usual. Every run ends with the `Goodbye.` line. The cells show only the other lines.

| Test | First run input lines | Second run input lines | Expected output | Actual output | Pass |
|---|---|---|---|---|---|
| 11 | Sign up Nimali, Log in Nimali, Create empty, `2`, `5` | None | `Account created.`, `Welcome, nimali.`, `Title must not be empty.`, `No documents.`<br>File: none | | |
| 12 | Sign up Nimali, Log in Nimali, Create Payroll, Create empty, Create Kandy rent, `2`, `5` | None | `Account created.`, `Welcome, nimali.`, `Saved document 1.`, `Title must not be empty.`, `Saved document 2.`, `1. Payroll`, `2. Kandy rent`<br>File: `[dictionary_1, dictionary_2]` | | |
| 13 | Sign up Nimali, Log in Nimali, `1`, empty line, `Body text`, `x`, `30`, `3`, `5` | None | `Account created.`, `Welcome, nimali.`, `Please enter a whole number.`, `Key must be from 1 to 25.`, `Title must not be empty.`<br>File: none | | |
| 14 | Sign up Nimali, Log in Nimali, Create Payroll, `3`, `2`, `3`, `0`, `5` | None | `Account created.`, `Welcome, nimali.`, `Saved document 1.`, `No such document.` twice<br>File: `[dictionary_1]` | | |
| 15 | Sign up Nimali, Log in Nimali, Create Payroll, `5` | Log in Nimali, `4`, `pay`, `3`, `1`, `3`, `5` | `Welcome, nimali.`, `1. Payroll`, `Salaries go out on 28 November.`<br>File: `[dictionary_1]` | | |
| 16 | `import vault` | None | Reads no input and prints nothing<br>File: none | | |
| 17 | Each Session 05 program test, 10 to 23 | As in the Session 05 test | As in the Session 05 test | | |

"File" is `documents.json` after the last run. In every program test, the program ends after `Goodbye.` and shows no Python error message.

## Done criteria

- ☐ Every test plan row gives exactly the expected result.
- ☐ The script runs from the terminal with `python vault.py`.

# Part 3 - Challenge: change history in models.py

## Goal

Make a document remember every change to its title. Save each change as the old title and the new title, and let `revert` undo the last change. Never save a change that fails, so the history holds only real changes.

Open the Python shell in the `06-employee-and-document-classes` folder and run `from models import *`, then make each document and run its steps in the test plan.

## Instructions

Amend your core `models.py`. Keep every core rule as it is, and add these steps to the `Document` class.

1. In the constructor, start the history as an empty list before the title is stored.
2. In the title setter, after the title passes the check, add the change to the history as a tuple `("title", old, new)`.
3. Define a method `revert` that takes no value.
   3.1. Raise `VaultError` with the message `Nothing to revert.` if the history is empty.
   3.2. Remove the last change from the history.
   3.3. Put the old title of that change back.

## Constraints

- The history is the attribute `history`, a list of tuples in the order the changes were made.
- After construction, `history` is an empty list. The title that the constructor stores adds nothing to it.
- Every successful title assignment adds one tuple, even when the new title equals the old one.
- A failed title assignment adds nothing.
- `revert` removes exactly one tuple and adds none.
- Every core test plan row still gives exactly the expected result.

## Design

This is advice on how to build the code well. No test checks it, and your explain-back looks at it.

- Every Design point in the core `models.py` test plan still applies.
- Start the history as an empty list before the constructor stores the title.
- Give every variable and function a type hint.

## Input validation

Not required beyond the core title rule.

## Test plan

Each test starts from a new `document`,
`Document(1, "Payroll", "Vdodulhv jr rxw rq 28 Qryhpehu.", "FRQWRVR-YDXOW")`. Each expected output gives `document.title`, then `document.history`.

| Test | Steps | Expected output | Actual output | Pass |
|---|---|---|---|---|
| 1 | None | `"Payroll"`<br>`[]` | | |
| 2 | Set the title to `"Payroll 2026"` | `"Payroll 2026"`<br>`[("title", "Payroll", "Payroll 2026")]` | | |
| 3 | Set the title to `"Payroll 2026"`, then to `"Salaries"` | `"Salaries"`<br>`[("title", "Payroll", "Payroll 2026"),`<br>`("title", "Payroll 2026", "Salaries")]` | | |
| 4 | Set the title to `"Payroll 2026"`, then to `"Salaries"`, then revert | `"Payroll 2026"`<br>`[("title", "Payroll", "Payroll 2026")]` | | |
| 5 | Set the title to `"Payroll 2026"`, then to `"Salaries"`, then revert twice | `"Payroll"`<br>`[]` | | |
| 6 | Revert | Raises `VaultError` with the message `Nothing to revert.` The title stays `"Payroll"`<br>`[]` | | |
| 7 | Set the title to `""` | Raises `ValidationError`. The title stays `"Payroll"`<br>`[]` | | |
| 8 | Set the title to `"Payroll 2026"`, then to `""` | Raises `ValidationError`. The title stays `"Payroll 2026"`<br>`[("title", "Payroll", "Payroll 2026")]` | | |
| 9 | Set the title to `"Payroll"` | `"Payroll"`<br>`[("title", "Payroll", "Payroll")]` | | |
| 10 | Set the title to `"Payroll 2026"`, revert, then revert again | Raises `VaultError` with the message `Nothing to revert.` The title stays `"Payroll"`<br>`[]` | | |

## Done criteria

- ☐ Every test plan row gives exactly the expected result.
- ☐ Every core `models.py` test plan row still gives exactly the expected result.